

TECHNICAL PROJECT REPORT

Low-Power Configurable Multi-Clock Digital System

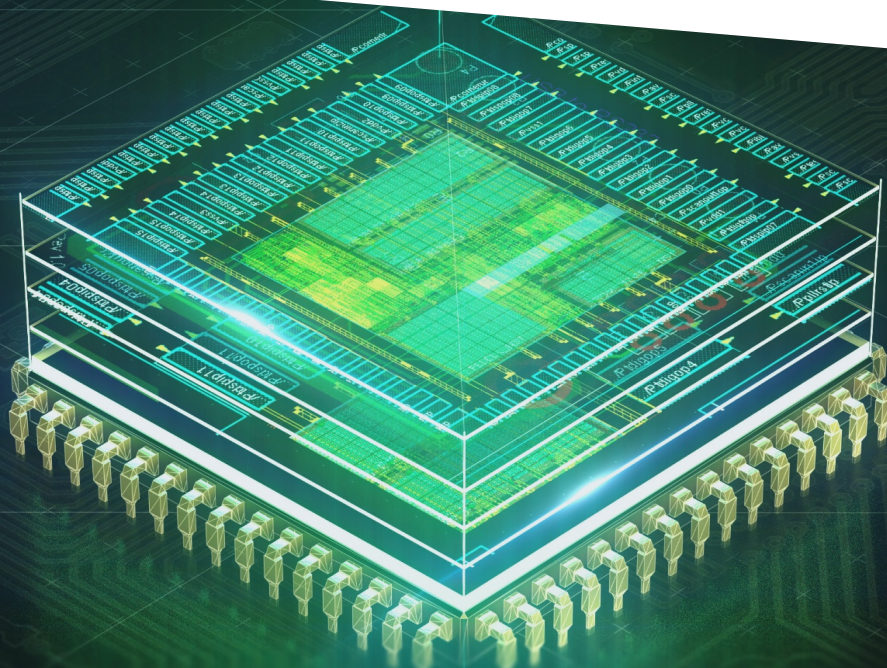
RTL Design • Lint & CDC Verification • Synthesis • DFT
Physical Implementation • Gate-Level Simulation
Timing & Power Signoff

Prepared by

Eng. Mohamed Elsayed Mohamed

Track: Digital IC Design

Instructor / Supervisor: Eng. Ali M. Eltemsah



Abstract

This report covers a UART-controlled low-power digital system with multiple clock domains. Commands enter through UART RX, the system controller decodes them, and the requested Register File or ALU operation is executed before the result is returned through UART TX. Synchronizers and an asynchronous FIFO are used on the CDC paths, and the ALU clock is gated when the block is idle.

The work started with RTL integration and functional verification, followed by SpyGlass lint/CDC checks, synthesis, Formality, DFT, and physical implementation. The final stages were post-PnR Formality, gate-level simulation with SDF back-annotation, and PrimeTime/PrimeTime PX timing and power analysis. The routed design finished with no setup or hold violations, and the geometry, connectivity and antenna checks were clean.

Contents

1	Final System Overview	1
1.1	Main Building Blocks	1
1.2	System Clocking and Configuration	1
2	Supported Command Protocol	2
3	UART Transmitter	3
4	UART Receiver	4
5	Asynchronous FIFO	6
6	Arithmetic Logic Unit	7
7	Register File	8
8	Synchronization Structures	9
8.1	Data Synchronizer	9
8.2	Reset Synchronizer	9
9	Clock Gating and Pulse Generation	10
9.1	Clock Gating	10
9.2	Pulse Generator	10
10	Clock Divider and System Controller	11
10.1	Clock Divider	11
10.2	System Controller	11
11	Functional Verification	12
12	SpyGlass Lint and CDC Verification	13
13	Logic Synthesis	14
13.1	Post-Synthesis Formal Verification	14
14	DFT Preparation	15
14.1	Post-DFT Formal Verification	15
15	Physical Implementation	16
15.1	Utilization and Density	16
15.2	Timing Closure	16
15.3	Final Physical Checks	17

16 Post-PnR Formality	18
17 Post-PnR Gate-Level Simulation	19
18 PrimeTime Timing and Power Signoff	21
19 Conclusion	22

List of Figures

1	Top-level architecture of the final multi-clock system.	1
2	UART transmitter internal architecture.	3
3	UART TX frame sequencing and busy behavior.	3
4	UART receiver internal architecture.	4
5	Illustrative UART RX oversampling timing for 4x, 8x and 16x sampling.	5
6	Asynchronous FIFO architecture used for safe multi-clock data transfer.	6
7	Integrated ALU block interface.	7
8	Register File block interface.	8
9	Multi-bit data synchronizer and destination-domain pulse generation.	9
10	Parameterized reset synchronizer.	9
11	Clock-gating block used for the ALU clock.	10
12	Pulse-generator logic.	10
13	Clock-divider block interface.	11
14	System controller interface.	11
15	System-level functional verification for the four supported commands.	12
16	SpyGlass lint, clock/reset and CDC checks reviewed during the project.	14
17	Synthesized <code>SYS_TOP</code> schematic generated by Synopsys Design Compiler.	14
18	Final synthesis constraint report: no violated constraints.	14
19	RTL changes used to prepare the multi-clock design for DFT insertion.	15
20	Gate-level design view after DFT insertion.	15
21	Final routed PnR layout of <code>SYS_TOP</code> in Cadence Encounter.	17
22	Post-PnR Formality result: Verification Succeeded.	18
23	Successful compilation of the post-PnR netlist and gate-level testbench in QuestaSim.	19
24	Post-PnR GLS UART transactions and expected responses.	20
25	Final post-PnR gate-level regression result: TEST PASSED	20
26	PrimeTime setup and hold violation reports.	21
27	PrimeTime PX time-based power report.	21

1 Final System Overview

The design uses two main clock domains. The REF_CLK domain contains the Register File, ALU, clock-gating logic and system controller. The UART side contains UART RX, UART TX, the pulse generator and the clock dividers. Synchronizers and the asynchronous FIFO connect the domains. The main transaction path is:

The main transaction path is:

UART RX → Data Synchronizer → SYS CTRL → RegFile / ALU → Async FIFO → UART TX

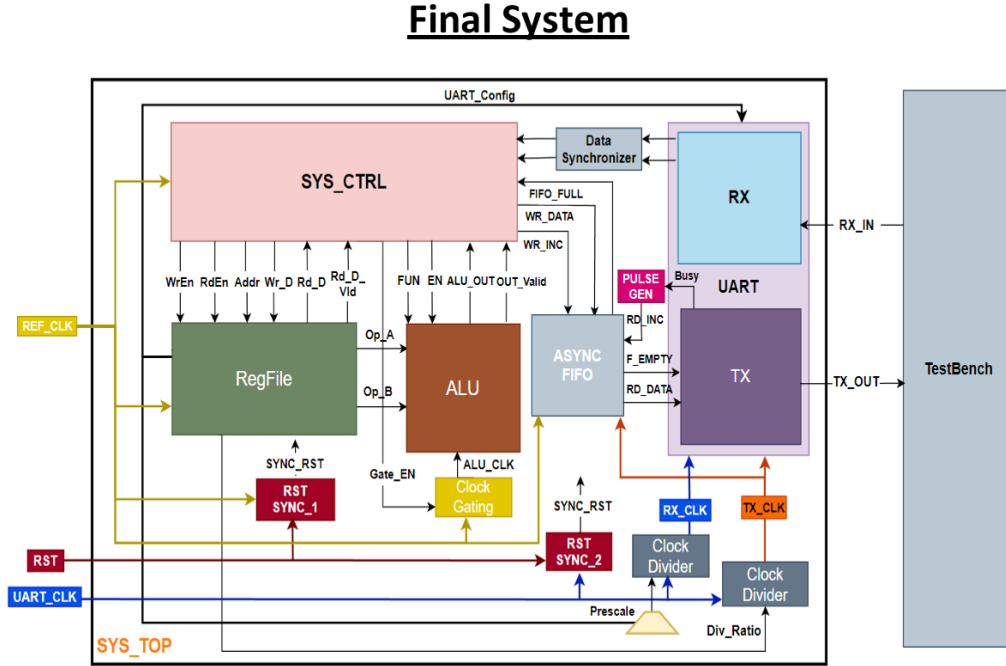


Figure 1: Top-level architecture of the final multi-clock system.

1.1 Main Building Blocks

Clock / Function Group	Blocks
REF_CLK domain	Register File, ALU, Clock Gating, System Controller
UART side	UART TX, UART RX, Pulse Generator, Clock Dividers
CDC / reset structures	Data Synchronizer, Reset Synchronizer, Asynchronous FIFO

Table 1: Functional partitioning of the system.

1.2 System Clocking and Configuration

The reference clock is 50 MHz and the UART clock is 3.6864 MHz. UART parity, parity type and oversampling settings are stored in the reserved Register File configuration locations, together with the programmable clock-divider ratio.

2 Supported Command Protocol

The UART interface supports four commands. UART RX receives the command and its data bytes, then the system controller selects the required Register File or ALU operation.

Command	Frames	Function
0xAA	3	Register File write: command, address and data byte.
0xBB	2	Register File read: command and address; the read value is returned through UART TX.
0xCC	4	ALU operation with new operands: command, Operand A, Operand B and ALU function.
0xDD	2	ALU operation without new operands: command and ALU function using already stored operands.

Table 2: Supported system commands.

Normal Register File read/write operations use addresses 0x04 through 0x0F, while addresses 0x00 through 0x03 are reserved for ALU operands and system configuration.

3 UART Transmitter

UART TX serializes an 8-bit data byte into a UART frame. A new byte is accepted when `Data_Valid` is asserted. While a frame is being sent, `Busy` remains high and no new byte is accepted. The line stays high in the idle state, and parity can be disabled, even or odd.

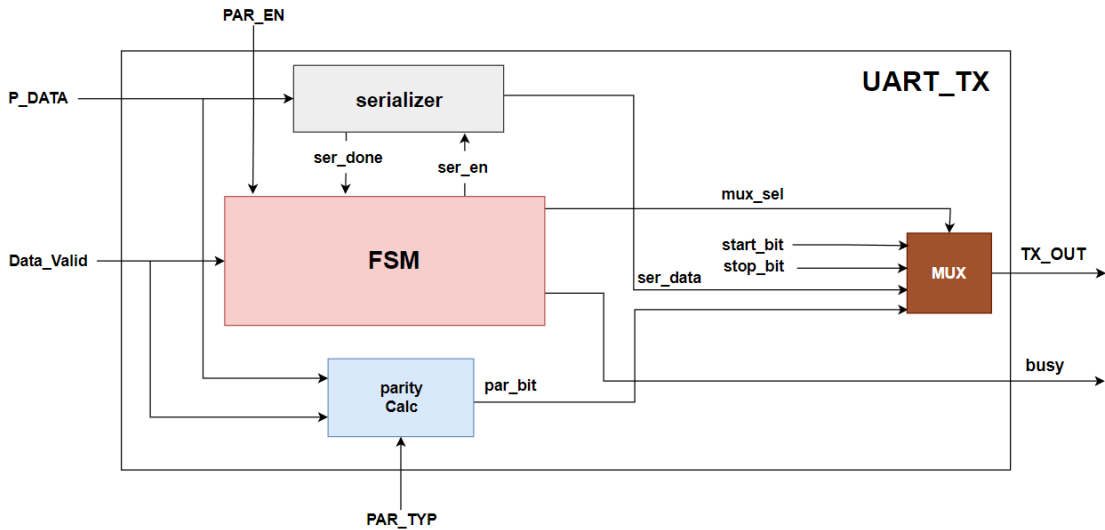


Figure 2: UART transmitter internal architecture.

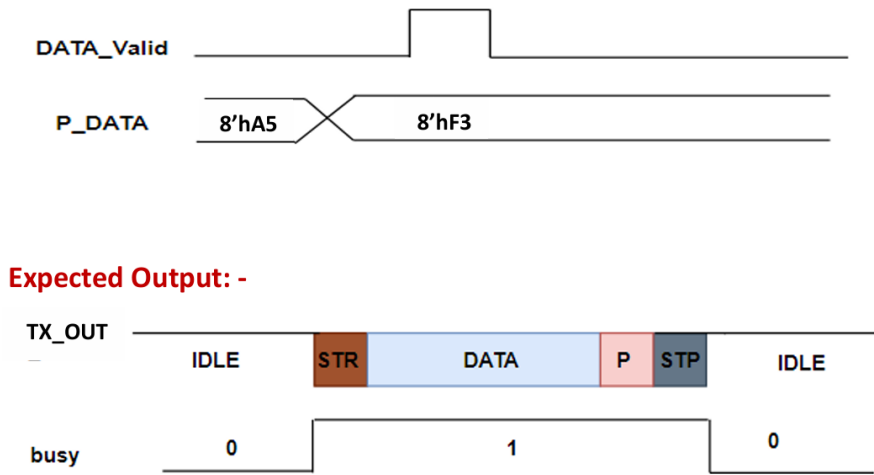


Figure 3: UART TX frame sequencing and busy behavior.

4 UART Receiver

UART RX reconstructs each serial frame into an 8-bit data byte. It checks the configured parity and stop bit, reports parity or framing errors when needed, and asserts `data_valid` for a valid frame.

The receiver supports 4x, 8x and 16x oversampling. The higher RX clock rate provides samples around the center of each UART bit, where the data is expected to be most stable.

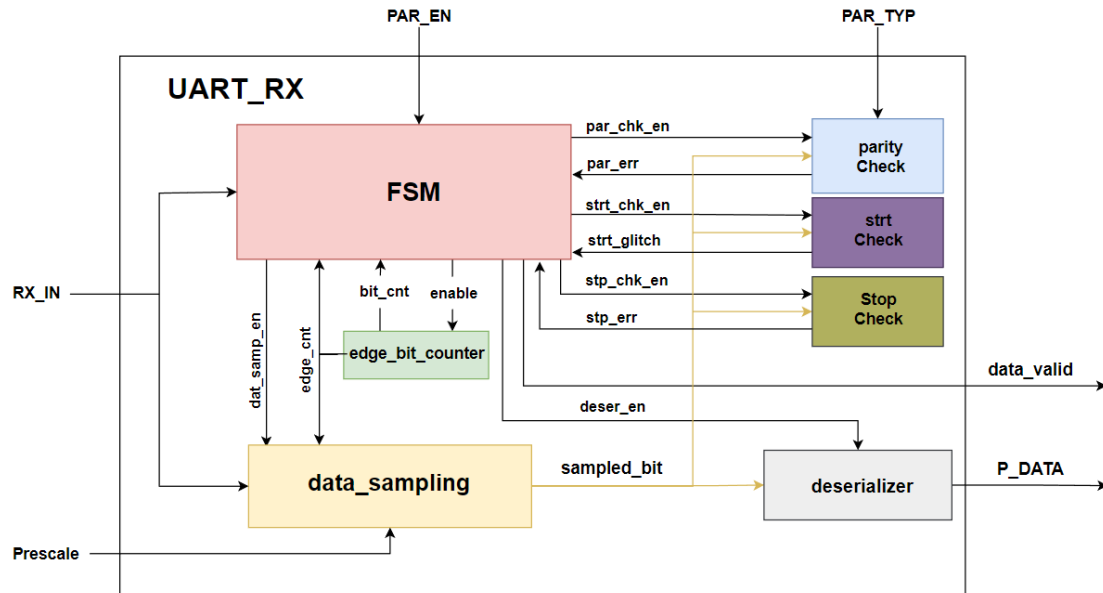
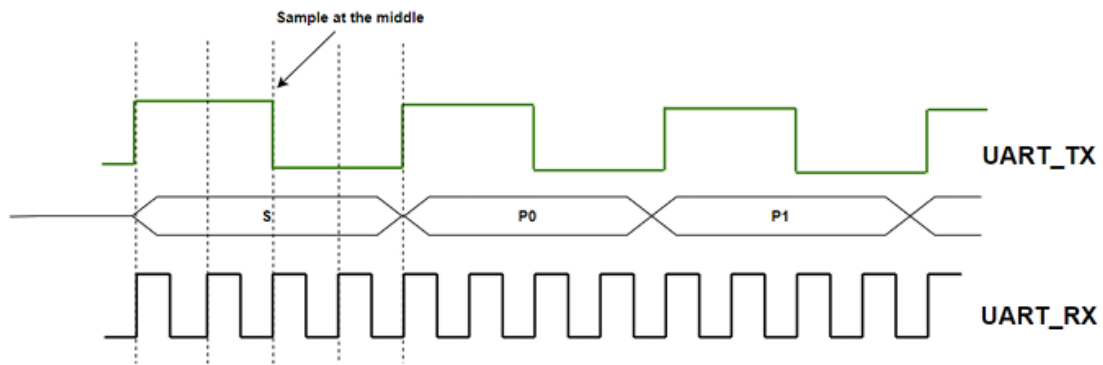


Figure 4: UART receiver internal architecture.

1. Oversampling by 4: This means that the clock speed of UART_RX is 4 times the speed of UART_TX.



2. Oversampling by 8: This means that the clock speed of UART_RX is 8 times the speed of UART_TX.

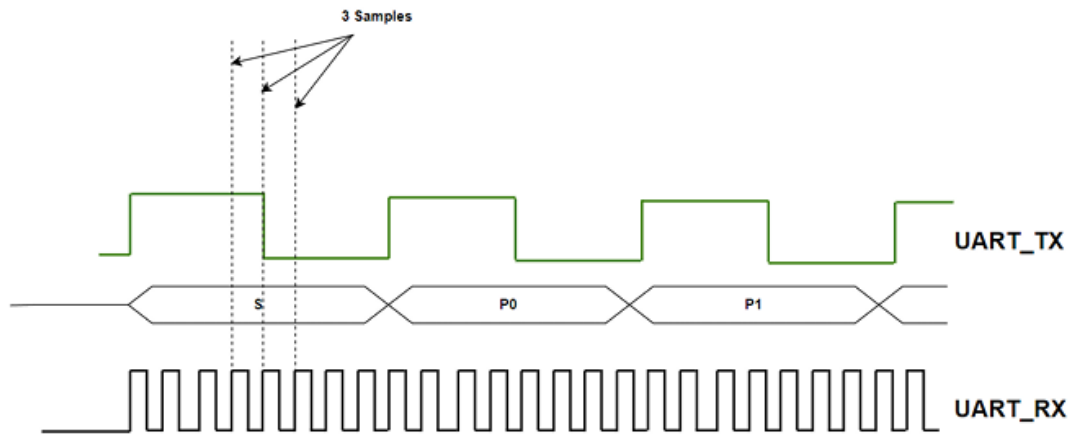


Figure 5: Illustrative UART RX oversampling timing for 4x, 8x and 16x sampling.

5 Asynchronous FIFO

The asynchronous FIFO carries result data between the two clock domains. The write and read sides use separate clocks and separate pointers. Gray-coded pointer information is synchronized across the boundary and is used to generate the full and empty flags, while the payload is stored in dual-port memory.

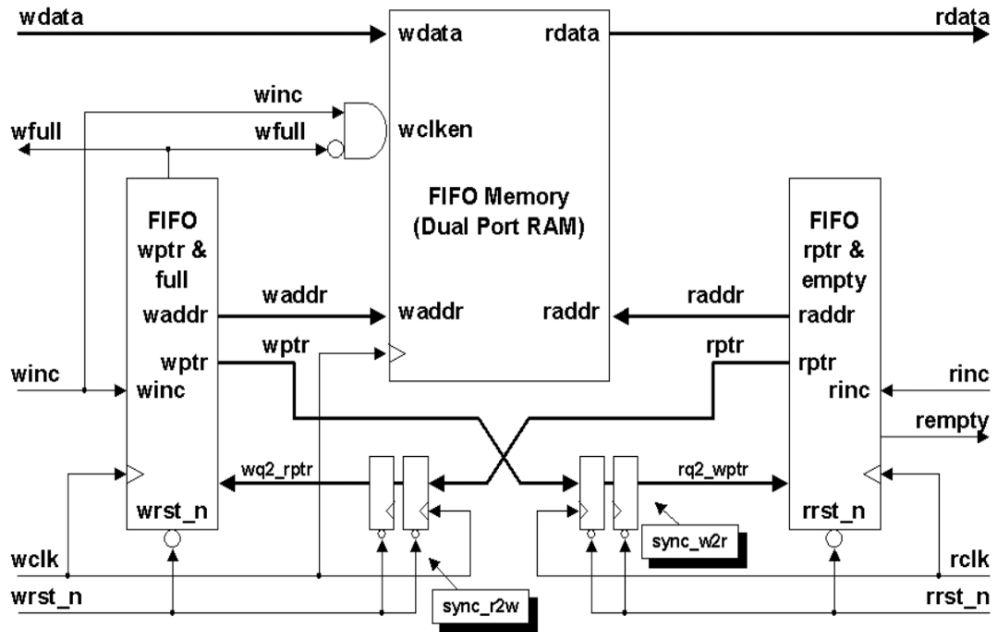


Figure 6: Asynchronous FIFO architecture used for safe multi-clock data transfer.

6 Arithmetic Logic Unit

The ALU is controlled by ALU_FUN and supports arithmetic, logic, comparison and shift operations. Its operands come from the reserved Register File locations. The controller enables the ALU only when an ALU command is active.

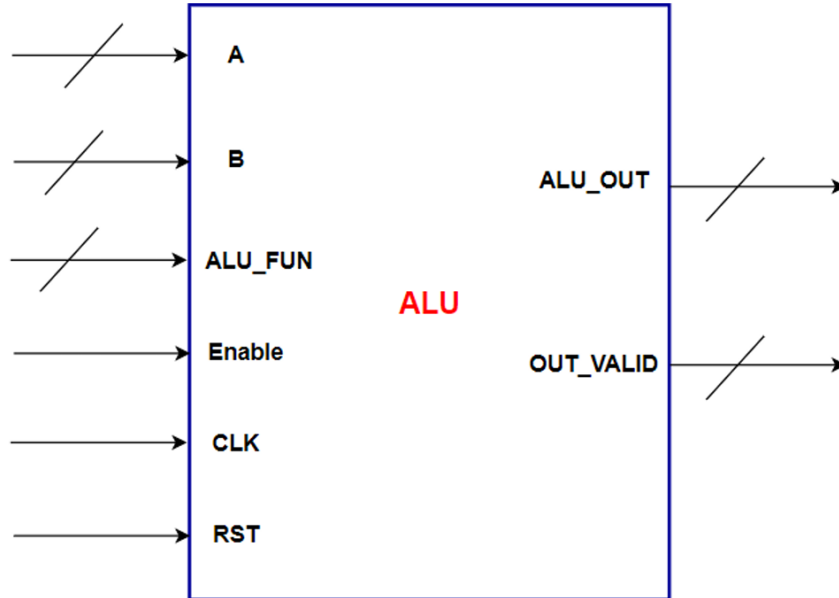


Figure 7: Integrated ALU block interface.

Code	Operation	Code	Operation
0000	Unsigned addition	1000	XOR
0001	Unsigned subtraction	1001	XNOR
0010	Unsigned multiplication	1010	Compare A = B
0011	Unsigned division	1011	Compare A > B
0100	AND	1100	Compare A < B
0101	OR	1101	Shift A right
0110	NAND	1110	Shift A left
0111	NOR	–	Reserved / default

Table 3: ALU operation encoding.

7 Register File

The Register File stores normal data as well as the ALU operands and system configuration. Addresses 0x00 to 0x03 are reserved for these special functions.

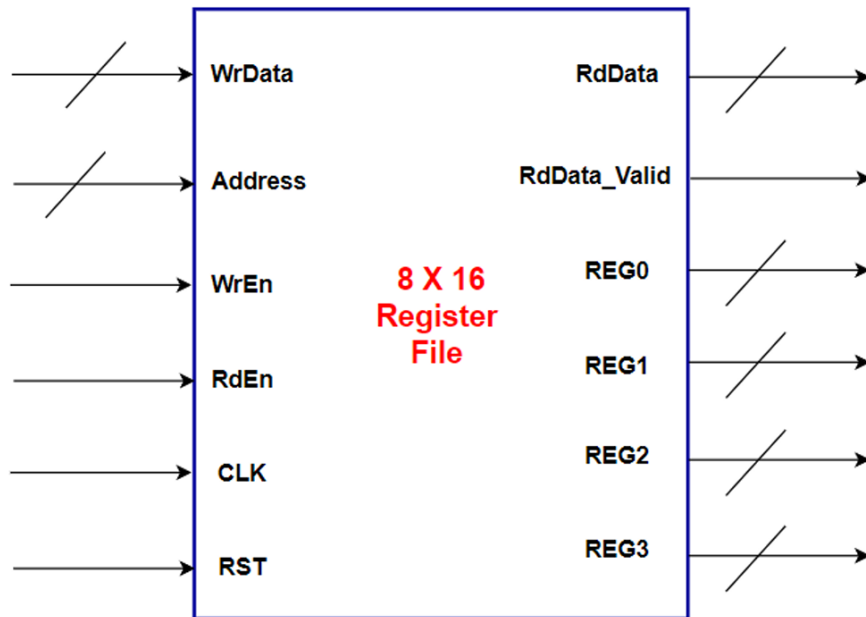


Figure 8: Register File block interface.

Address	Reserved function
0x00	ALU Operand A
0x01	ALU Operand B
0x02	UART configuration: parity enable, parity type and prescale
0x03	Clock-divider ratio

Table 4: Reserved Register File locations.

8 Synchronization Structures

8.1 Data Synchronizer

The data synchronizer moves a multi-bit bus between clock domains by synchronizing the control signal rather than every data bit. The source enable passes through a multi-flop synchronizer and is converted into a pulse in the destination domain, which is then used to sample the bus.

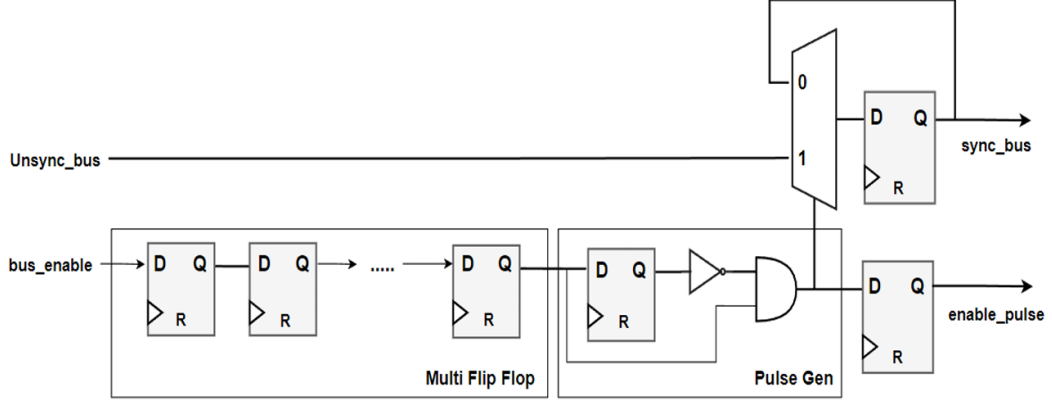


Figure 9: Multi-bit data synchronizer and destination-domain pulse generation.

8.2 Reset Synchronizer

The reset synchronizer keeps reset assertion asynchronous but releases reset in sync with the destination clock. This avoids releasing reset close to an active clock edge.

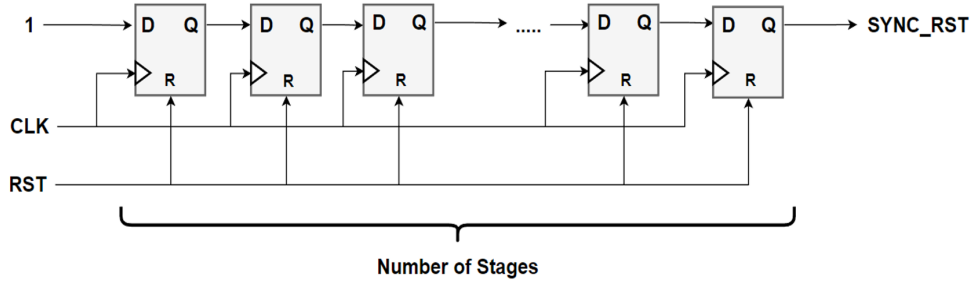


Figure 10: Parameterized reset synchronizer.

9 Clock Gating and Pulse Generation

9.1 Clock Gating

The ALU clock is turned on only when an ALU operation is requested. The clock-gating latch holds the enable stable while the source clock is high, which prevents glitches and avoids unnecessary ALU switching while the block is idle.



Figure 11: Clock-gating block used for the ALU clock.

9.2 Pulse Generator

The pulse generator converts an enable transition into a one-cycle pulse in the destination clock domain. This pulse is used by the CDC control path.

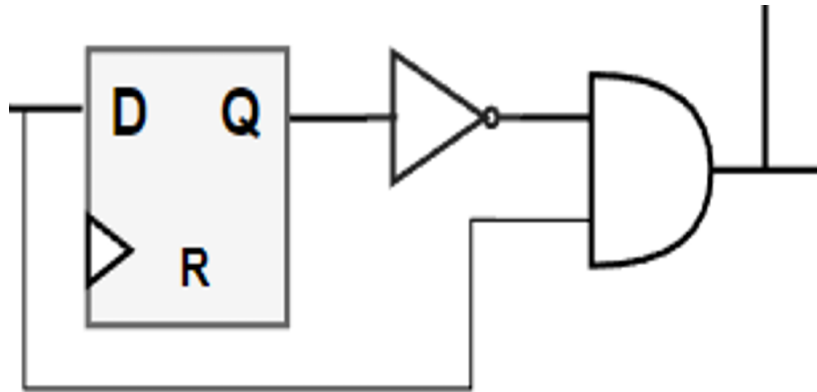


Figure 12: Pulse-generator logic.

10 Clock Divider and System Controller

10.1 Clock Divider

The clock divider generates the UART-derived clocks using a programmable division ratio. The ratio is stored in reserved Register File location REG3.

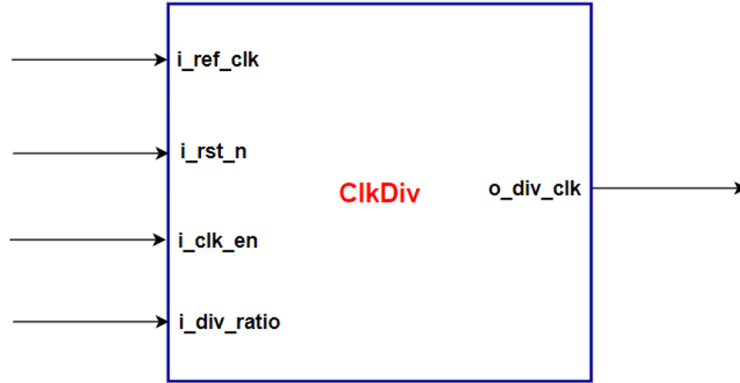


Figure 13: Clock-divider block interface.

10.2 System Controller

The system controller is the main command FSM. It decodes the received UART command, controls Register File reads and writes, starts ALU operations, and forwards valid results to the UART TX path through the asynchronous FIFO.

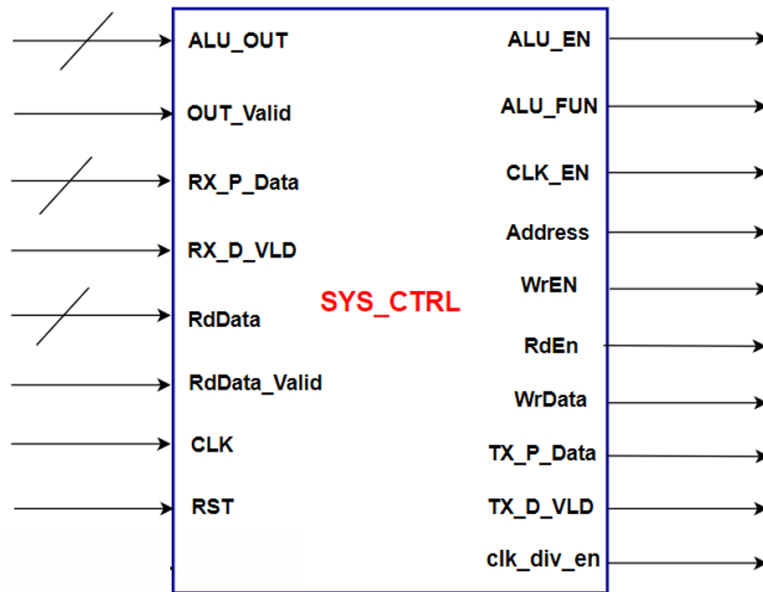


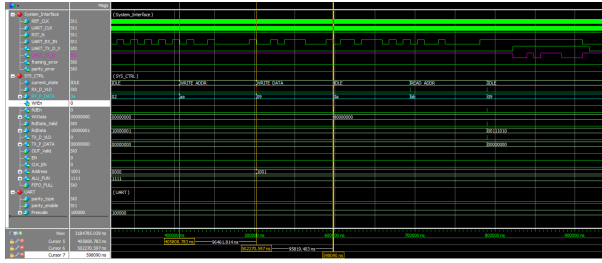
Figure 14: System controller interface.

11 Functional Verification

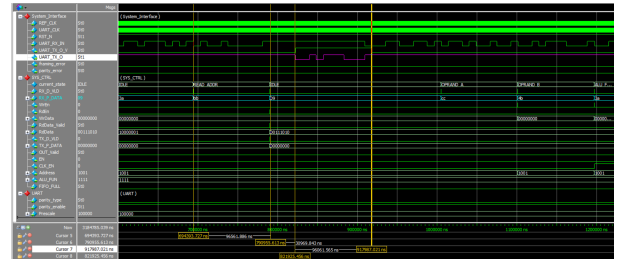
A self-checking testbench was used to run complete UART transactions through the system. The test cases cover Register File write, Register File read, an ALU operation with new operands, and an ALU operation using the previously stored operands. The reserved configuration registers were also exercised.

Test	Command	Example sequence
1	Register File Write	AA -> 09 -> 3A
2	Register File Read	BB -> 09
3	ALU with operands	CC -> 4B -> 3A -> 02
4	ALU without new operands	DD -> 04

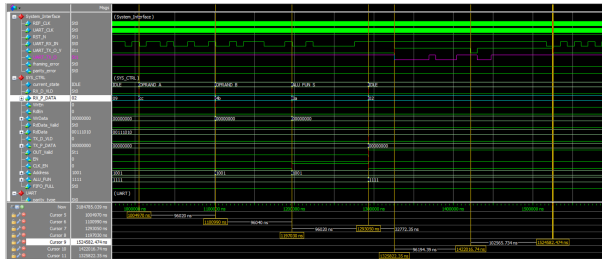
Table 5: Main functional test cases.



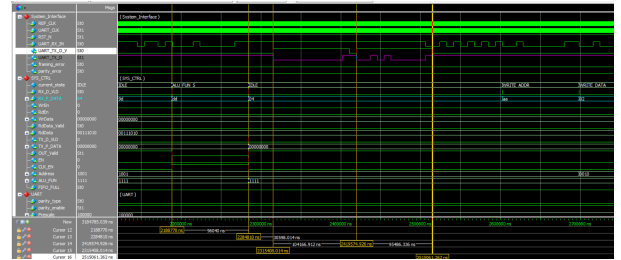
(a) Register File write command.



(b) Register File read command.



(c) ALU command with operands.



(d) ALU command without new operands.

Figure 15: System-level functional verification for the four supported commands.

12 SpyGlass Lint and CDC Verification

SpyGlass was run before synthesis to check lint, clock/reset integrity and CDC paths. Both functional CDC checks and structural CDC checks were included.

The lint run produced seven messages: three informational messages, one error and three warnings. The four non-informational messages were reviewed. They correspond to the intentional clock-gating latch, the two RX/TX clock-divider toggle warnings, and the counter logic that was classified as FSM-like logic. Three waiver rules cover those four messages because the same clock-divider waiver applies to both divider instances.

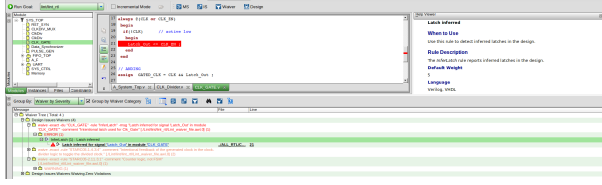
In Table 6, **Generated** is the total number of messages from the goal, **Waived** is the number removed by accepted waivers, and **Reported** is what remains after the waivers are applied. **Applied Waivers** counts the waiver rules that matched; one rule can match more than one message.

Goal / Report	Generated	Waived	Reported	Applied Waivers
Lint / lint_rtl	7	4	3	3
CDC Verify	58	5	53	4
CDC Verify Struct	50	1	49	1
Clock / Reset Integrity	11	2	9	1

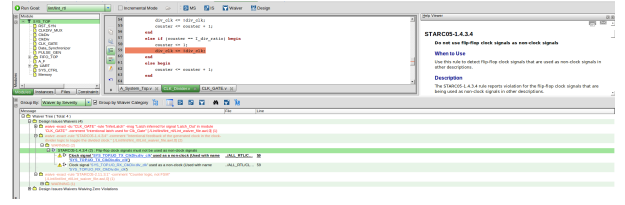
Table 6: SpyGlass message and waiver statistics.

Rule	Reviewed Scope	Engineering Justification
InferLatch	CLK_GATE	Intentional level-sensitive latch used by the glitch-free clock-gating structure to keep the clock-enable value stable while the source clock is high.
STARC05-1.4.3.4	RX / TX Clock Dividers	The divider output is intentionally fed back as data using <code>div_clk <= !div_clk</code> . The same intentional toggle structure exists in both the RX and TX clock dividers.
STARC05-2.11.3.1	Edge / Bit Counter	The reported block is counter/control logic rather than a finite-state machine. The rule is triggered because sequential and conditional control logic are implemented in the same always block.
Clock_glitch04	UART Clock-Control Paths	Prescale and division-ratio configuration values remain static while their corresponding UART clocks are active.
Ac_conv04	Asynchronous FIFO	FIFO payload data is protected by synchronized Gray-coded pointer and read-control logic rather than independent synchronizers on every payload bit.

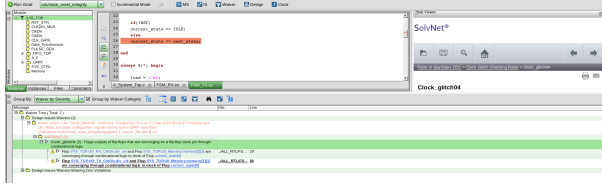
Table 7: Reviewed SpyGlass findings and engineering justification.



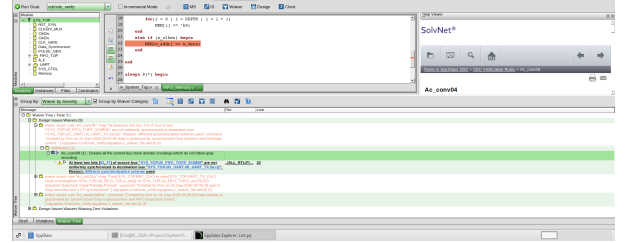
(a) Intentional clock-gating latch review.



(b) Clock-divider toggle logic review.



(c) Clock/reset integrity review.

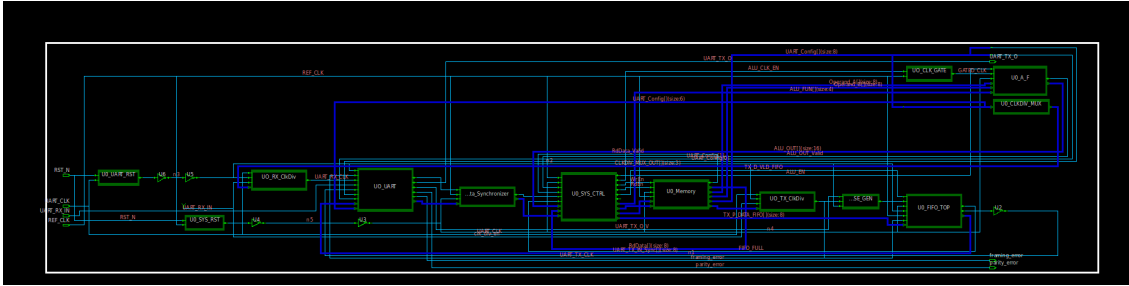


(d) CDC review for the asynchronous FIFO path.

Figure 16: SpyGlass lint, clock/reset and CDC checks reviewed during the project.

13 Logic Synthesis

The RTL was synthesized in Synopsys Design Compiler using the project timing constraints. The synthesized schematic shows the gate-level structure of `SYS_TOP`, and the final constraint report shows no violated constraints.

**Figure 17:** Synthesized `SYS_TOP` schematic generated by Synopsys Design Compiler.

```
*****
Report : constraint
        -all_violators
Design : SYS_TOP
Version: 0-2018.06-SP1
Date   : Tue Oct 6 05:41:54 2026
*****
```

This design has no violated constraints.

Figure 18: Final synthesis constraint report: no violated constraints.

13.1 Post-Synthesis Formal Verification

The RTL reference was compared with the synthesized netlist in Synopsys Formality.

Post-Synthesis Formality Result: Verification Succeeded

14 DFT Preparation

DFT logic was added to make the sequential state controllable and observable in scan mode. The top level was extended with `scan_clk`, `scan_rst`, `test_mode`, `SE`, `SI` and `SO`. In test mode, the internal clock domains can be driven from the scan clock and the reset path can be driven from the scan reset.

```

input          scan_clk,
input          scan_rst,
input          test_mode,
input          SE,
input [NUM_OF_CHAINS-1:0] SI,
output [NUM_OF_CHAINS-1:0] SO,

```

```

wire SYS_DFT_CLK;
wire UART_DFT_CLK;
wire RX_DFT_CLK;
wire TX_DFT_CLK;

assign SYS_DFT_CLK = test_mode ? scan_clk : REF_CLK;
assign UART_DFT_CLK = test_mode ? scan_clk : UART_CLK;
assign RX_DFT_CLK = test_mode ? scan_clk : UART_RX_CLK;
assign TX_DFT_CLK = test_mode ? scan_clk : UART_TX_CLK;

wire DFT_RST;
wire SYS_DFT_RST;
wire UART_DFT_RST;

assign DFT_RST = test_mode ? scan_rst : RST_N;
assign SYS_DFT_RST = test_mode ? scan_rst : SYNC_SYS_RST;
assign UART_DFT_RST = test_mode ? scan_rst : SYNC_UART_RST;

wire ALU_CLK_EN_TEST;

assign ALU_CLK_EN_TEST = ALU_CLK_EN | test_mode;

```

(a) Top-level scan interface added during DFT preparation.

(b) Test-mode clock/reset selection and ALU clock-enable adaptation.

Figure 19: RTL changes used to prepare the multi-clock design for DFT insertion.

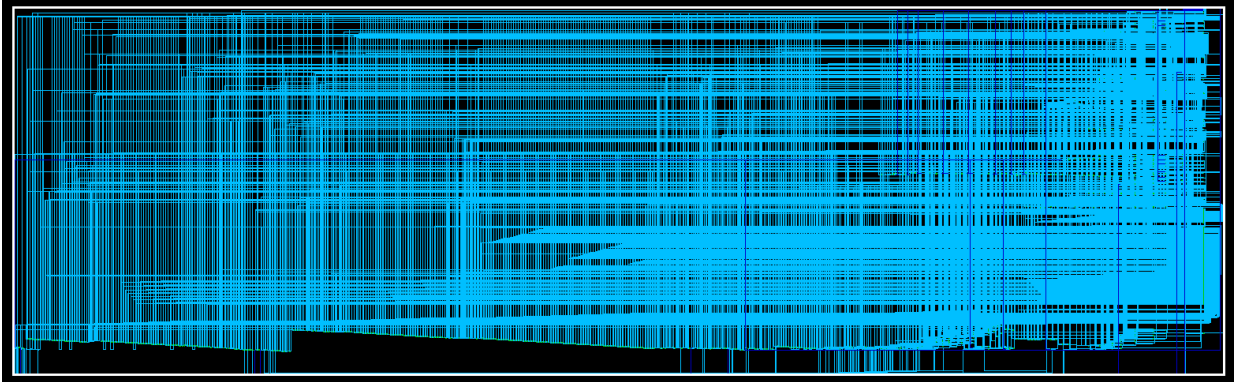


Figure 20: Gate-level design view after DFT insertion.

14.1 Post-DFT Formal Verification

The post-DFT netlist was checked again against the reference design in Synopsys Formality.

Post-DFT Formality Result: **Verification Succeeded**

15 Physical Implementation

The DFT netlist was implemented in Cadence Encounter through floorplanning, placement, CTS, timing optimization, routing and chip finishing. MMMC views were set up for functional, scan and capture modes. The slow library was used for setup analysis and the fast library for hold analysis.

15.1 Utilization and Density

Stage	Metric	Value
After placement	Core utilization	63.16%
After placement	Cell utilization	62.97%
After CTS	Core utilization	67.62%
After CTS	Cell utilization	67.42%
Post-route	Placement density	68.17%

Table 8: Physical implementation utilization progression.

15.2 Timing Closure

After CTS, hold timing was the main issue: the worst hold slack was -0.708 ns with 341 violating paths. Post-CTS optimization removed the hold violations without breaking setup timing, and the routed design kept positive setup and hold slack.

Stage	Setup WNS	Hold WNS	Hold TNS	Hold Violations
After CTS	+0.305 ns	-0.708 ns	-64.169 ns	341
CTS + optimization	+0.201 ns	+0.027 ns	0.000 ns	0
Post-route	+0.255 ns	+0.037 ns	0.000 ns	0

Table 9: Timing evolution through CTS, optimization and routing.

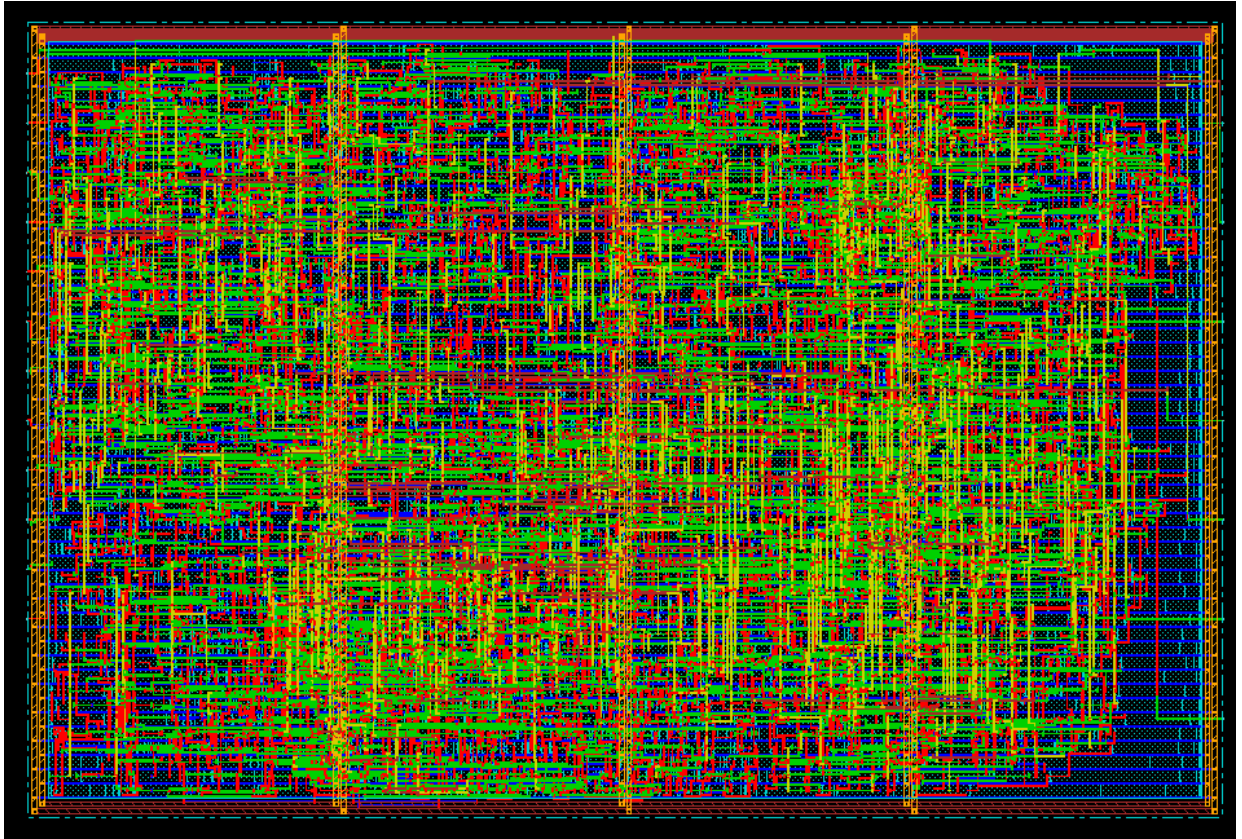


Figure 21: Final routed PnR layout of SYS_TOP in Cadence Encounter.

15.3 Final Physical Checks

Check	Final result
Maximum capacitance / transition / fanout	0 / 0 / 0 violations
Geometry / DRC	0 violations
Connectivity	0 violations, 0 warnings
Process antenna	0 violations

Table 10: Final post-route physical verification results.

16 Post-PnR Formality

Formality was run again after PnR using the routed implementation as the implementation side. The check finished with **Verification Succeeded** and 362 passing points.

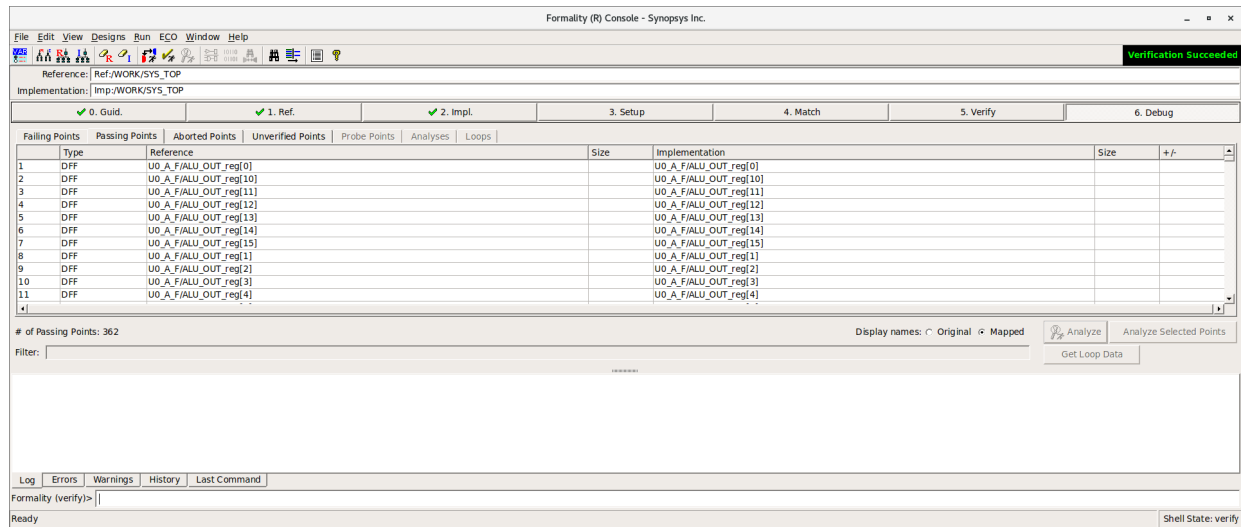


Figure 22: Post-PnR Formality result: Verification Succeeded.

17 Post-PnR Gate-Level Simulation

The routed gate-level netlist was simulated in QuestaSim with the post-PnR SDF. The GLS testbench was adjusted for the gate-level hierarchy and the added DFT ports, then the UART command tests were run again. A VCD file was recorded from the same simulation for PrimeTime PX.

```
QuestaSim-64 vlog 10.7c Compiler 2018.08 Aug 17 2018
Start time: 06:00:42 on Oct 06,2026
vlog -reportprogress 300 -sv ../PNR_Results/SYS_TOP.v
-- Compiling module SYS_TOP
-- Compiling module CLK_GATE
-- Compiling module ClkDiv_DIV_RATIO_WIDTH3_test_1
-- Compiling module ClkDiv_DIV_RATIO_WIDTH8_test_1

Top level modules:
    SYS_TOP
End time: 06:00:42 on Oct 06,2026, Elapsed time: 0:00:00
Errors: 0, Warnings: 0
QuestaSim-64 vlog 10.7c Compiler 2018.08 Aug 17 2018
Start time: 06:00:42 on Oct 06,2026
vlog -reportprogress 300 -sv A_System_TB.sv
-- Compiling module tb_SYS_TOP

Top level modules:
    tb_SYS_TOP
End time: 06:00:42 on Oct 06,2026, Elapsed time: 0:00:00
Errors: 0, Warnings: 0
```

Figure 23: Successful compilation of the post-PnR netlist and gate-level testbench in QuestaSim.

[0] Testbench started in Functional Mode

[1765000] Reset released and synchronized

[1765000] UART RX Send = 0xbb

[97792000] UART RX Send = 0x02

[314535000] UART TX Captured = 0x81

PASS: Expected = 0x81, Received = 0x81

Response started after 29572000

(a) Register File read response check.

[314535000] UART RX Send = 0xaa

[410564000] UART RX Send = 0x09

[506593000] UART RX Send = 0x3a

[602622000] UART RX Send = 0xbb

[698651000] UART RX Send = 0x09

[913495000] UART TX Captured = 0x3a

PASS: Expected = 0x3a, Received = 0x3a

Response started after 27673000

(b) Register File write/read response check.

[913495000] UART RX Send = 0xcc

[1009524000] UART RX Send = 0x4b

[1105553000] UART RX Send = 0x3a

[1201582000] UART RX Send = 0x02

[1416968000] UART TX Captured = 0x10

PASS: Expected = 0x10, Received = 0x10

Response started after 28216000

(c) ALU command response check.

[1521135000] UART RX Send = 0xaa

[1617164000] UART RX Send = 0x00

[1713193000] UART RX Send = 0xab

[1809222000] UART RX Send = 0xaa

[1905251000] UART RX Send = 0x01

[2001280000] UART RX Send = 0x9d

[2097309000] UART RX Send = 0xdd

[2193337000] UART RX Send = 0x04

[2406554000] UART TX Captured = 0x00

PASS: Expected = 0x00, Received = 0x00

Response started after 26046000

(d) Extended UART command regression.

Figure 24: Post-PnR GLS UART transactions and expected responses.

```

: =====
:          TEST PASSED
: =====
:

```

Figure 25: Final post-PnR gate-level regression result: **TEST PASSED**.

All GLS command tests passed with the expected UART responses after SDF back-annotation.

18 PrimeTime Timing and Power Signoff

PrimeTime was used for the final setup and hold checks with the exported post-route netlist, SDC and SDF. Both violation reports were clean. PrimeTime PX then read the VCD switching activity and ran time-based power analysis.

<pre>***** Report : timing -path_type full -delay_type max -slack_lesser_than 0.00 -max_paths 100 -sort_by slack Design : SYS_TOP Version: 0-2018.06-SP1 Date : Mon Oct 5 23:32:17 2026 ***** No paths with slack less than 0.00.</pre>	<pre>***** Report : timing -path_type full -delay_type min -slack_lesser_than 0.00 -max_paths 100 -sort_by slack Design : SYS_TOP Version: 0-2018.06-SP1 Date : Mon Oct 5 23:32:17 2026 ***** No paths with slack less than 0.00.</pre>
---	---

(a) Setup report: no path with slack below 0 ns.

(b) Hold report: no path with slack below 0 ns.

Figure 26: PrimeTime setup and hold violation reports.

```
*****
Report : Time Based Power
Design : SYS_TOP
Version: 0-2018.06-SP1
Date   : Mon Oct 5 23:32:17 2026
*****
```

Attributes

```
-----
i - Including register clock pin internal power
u - User defined power group
```

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)	Attrs
clock_network	2.453e-04	8.642e-05	2.555e-06	3.343e-04	(96.14%)	i
register	4.735e-07	7.169e-08	5.952e-06	6.497e-06	(1.87%)	
combinational	3.304e-07	1.660e-07	6.445e-06	6.942e-06	(2.00%)	
sequential	0.0000	0.0000	0.0000	0.0000	(0.00%)	
memory	0.0000	0.0000	0.0000	0.0000	(0.00%)	
io_pad	0.0000	0.0000	0.0000	0.0000	(0.00%)	
black_box	0.0000	0.0000	0.0000	0.0000	(0.00%)	
Net Switching Power	= 8.666e-05	(24.92%)				
Cell Internal Power	= 2.462e-04	(70.78%)				
Cell Leakage Power	= 1.495e-05	(4.30%)				

Total Power	= 3.478e-04	(100.00%)				
X Transition Power	= 0.0000					
Glitching Power	= 0.0000					
Peak Power	= 0.1540					
Peak Time	= 390220.833					

Figure 27: PrimeTime PX time-based power report.

Signoff item	Final result
PrimeTime setup violations	0
PrimeTime hold violations	0
Worst PrimeTime setup slack	+0.25 ns
Worst PrimeTime hold slack	+0.23 ns
VCD annotation	100% nets and 100% leaf cells annotated
Time-based total power	3.478×10^{-4} W

Table 11: PrimeTime / PrimeTime PX signoff summary.

19 Conclusion

This project covered the full implementation of the multi-clock UART-controlled system from RTL integration to backend signoff. The final design includes the UART path, Register File, configurable ALU, CDC structures, clock gating and asynchronous FIFO. Verification included SpyGlass lint/CDC, Formality after synthesis, DFT and PnR, post-PnR GLS with SDF, and PrimeTime timing/power checks.

The routed design finished with zero setup and hold violations. Geometry, connectivity and antenna checks were also clean, and the post-PnR GLS passed the same command set used during RTL verification.